Trabajo Final — Documento Final

Entrega 3: Divide y Vencerás, Programación Dinámica y Comparativa Final

Análisis y Diseño de Algoritmos — 2026-2

Caso de uso 3.4: Optimización de Tareas en Servidores

Integrantes

Valery Monsalve Correa

Samuel Uribe Salazar

Sección	Contenido
1	Problema, datos de prueba y resumen de las Entregas 1 y 2
2	Solución por Divide y Vencerás (dividir, vencer, combinar, Teorema Maestro)
3	Solución por Programación Dinámica
4	Comparativa final de las 5 técnicas (tabla, gráficas, análisis)
5	Conclusiones, escenarios reales y limitaciones
6	Ejecución, estructura del repositorio y referencias

1. EL PROBLEMA Y LO QUE LLEVAMOS

1.1 Definición

Un servidor tiene un tiempo total disponible **T** y recibe **n** tareas. Cada tarea *i* tiene un tiempo de ejecución $\mathbf{t_i}$ y una prioridad $\mathbf{p_i}$. Hay que elegir qué tareas ejecutar (cada una completa o nada: $x_i \in \{0, 1\}$) para **maximizar la prioridad total** sin que la suma de tiempos supere T. Formalmente es el **problema de la mochila 0/1**, un problema NP-difícil: no se conoce un algoritmo que lo resuelva en tiempo polinomial en el tamaño de la entrada.

Entrada: archivo de texto con la línea `n T` y luego una línea `id tiempo prioridad` por tarea. **Salida:** prioridad máxima, IDs de las tareas elegidas y tiempo usado.

Alcance: el caso 3.4 de la guía menciona varios servidores y dependencias. Desde la Entrega 1 lo acotamos a un servidor sin dependencias, que es el núcleo del problema (decidir qué tareas caben) y permite aplicar y comparar las 5 técnicas sobre el mismo modelo. La extensión a varios servidores se discute en la sección 5.3.

1.2 Datos de prueba

Todos los casos (salvo el pequeño, hecho a mano en la Entrega 1) se generan con `src/generar_datos.py` usando semilla fija, así que cualquiera puede reproducirlos. Los tiempos están entre 5 y 30, la prioridad es el tiempo más un ruido entre −3 y 12 (las tareas largas suelen ser más importantes) y T es el 45 % de la suma de tiempos. Para la Entrega 3 se agregaron los casos de 100 y 1000 tareas que pide la tabla comparativa.

Archivo	n	T	Óptimo	Uso
caso_contraejemplo_greedy.txt	3	10	10	Contraejemplo mínimo del greedy (Entrega 2)
caso_pequeno.txt	10	50	90	Verificación a mano (Entrega 1)
caso_mediano.txt	20	159	255	Semilla 202
caso_grande.txt	28	241	378	Semilla 391. Límite práctico de la fuerza bruta (46 s)
caso_n100.txt	100	775	1173	Semilla 100. Columna n=100 de la tabla final (nuevo)
caso_n1000.txt	1000	7636	11843	Semilla 1008. Columna n=1000 de la tabla final (nuevo)

1.3 Resumen de las Entregas 1 y 2

Técnica	Idea en una frase	Complejidad	¿Óptima?
Fuerza bruta (E1)	Probar todas las 2^n combinaciones de incluir/excluir cada tarea.	$O(2^n)$	Sí
Greedy (E2)	Ordenar por prioridad/tiempo y tomar lo que quepa. Nunca reconsidera.	O(n log n)	No
Backtracking (E2)	Árbol de la fuerza bruta, pero corta ramas que no caben o que no pueden superar la mejor solución (cota fraccional).	$O(n \cdot 2^n)$ peor caso	Sí

Los detalles de esas tres técnicas están en `docs/entrega1.pdf` y `docs/entrega2.pdf`. Este documento las incluye en la comparativa final junto con las dos técnicas nuevas.

2. SOLUCIÓN POR DIVIDE Y VENCERÁS (src/divide_y_venceras.py)

2.1 Cómo dividir el problema en subproblemas independientes

Se parte la lista de tareas en dos mitades. Las mitades son **independientes**: elegir tareas en la izquierda no cambia las tareas disponibles en la derecha. Lo único que comparten es la capacidad T, y eso se resuelve al combinar.

El reto: no basta con que cada mitad devuelva "su mejor prioridad". Si la mitad izquierda usa casi toda la capacidad para lograr su máximo, puede dejar sin espacio a la derecha, y la mejor solución global puede usar una opción "peor" de la izquierda que deja más tiempo libre. Por eso cada subproblema devuelve su **frente de Pareto**:

> **Frente de Pareto** = la lista de opciones (tiempo usado, prioridad obtenida) que vale la pena conservar. Una opción se descarta si otra usa **igual o menos tiempo** y da **igual o más prioridad** (está *dominada*). Por ejemplo, entre (6, 7) y (7, 6) se descarta (7, 6): gasta más y da menos. El frente queda ordenado por tiempo y con prioridad creciente, y nunca tiene más de T+1 opciones.

2.2 Las tres fases

Fase	Qué hace	Código
1. Dividir	Parte las tareas [inicio, fin) por la mitad.	medio = (inicio + fin) // 2
2. Vencer	Resuelve cada mitad recursivamente y obtiene dos frentes. **Caso base**: una tarea → {no tomarla (0, 0), tomarla (t, p) si cabe}.	izq = resolver_frente(...) der = resolver_frente(...)
3. Combinar	Suma cada opción de la izquierda con cada opción de la derecha que quepa en T, y deja solo las no dominadas.	combinar(izq, der, T)

2.3 Ejemplo paso a paso (contraejemplo del greedy, T = 10)

Tareas: 1 = (6, 7), 2 = (5, 5), 3 = (5, 5), escritas como (tiempo, prioridad).

```
DIVIDIR:  [T1]  |  [T2, T3]          luego  [T2] | [T3]

VENCER (casos base):
  [T1] -> {(0,0), (6,7)}       [T2] -> {(0,0), (5,5)}       [T3] -> {(0,0), (5,5)}

COMBINAR [T2] + [T3]:
  (0,0)+(0,0)=(0,0)  (0,0)+(5,5)=(5,5)  (5,5)+(0,0)=(5,5) repetida  (5,5)+(5,5)=(10,10)
  frente = {(0,0), (5,5), (10,10)}

COMBINAR [T1] + [T2,T3]:
  (0,0)+... = (0,0), (5,5), (10,10)     (6,7)+(0,0) = (6,7)     (6,7)+(5,5) = 11 > 10  NO CABE
  frente final = {(0,0), (5,5), (6,7), (10,10)}   -> la mejor es (10,10): tareas {2, 3}
```

El greedy se equivocaba aquí (tomaba la tarea 1 y obtenía 7). Divide y vencerás encuentra el 10 porque conserva la opción (5, 5) de la derecha aunque no sea "la mejor" por sí sola.

2.4 Implementación (núcleo)

```
def resolver_frente(tareas, inicio, fin, capacidad):
    if fin - inicio == 1:                                  # caso base
        return caso_base(tareas[inicio], capacidad)
    medio = (inicio + fin) // 2                            # 1. DIVIDIR
    izq = resolver_frente(tareas, inicio, medio, capacidad) # 2. VENCER
    der = resolver_frente(tareas, medio, fin, capacidad)
    return combinar(izq, der, capacidad)                   # 3. COMBINAR

def combinar(izq, der, capacidad):
    mejor_por_tiempo = {}
    for a in izq:
        for b in der:                      # der esta ordenado por tiempo
            if a[0] + b[0] > capacidad:
                break
            ... guardar la mayor prioridad para cada tiempo a[0] + b[0]
    ... recorrer por tiempo creciente y dejar solo las opciones no dominadas
```

Cada opción guarda una referencia a las dos opciones que la formaron. Al final, `reconstruir()` recorre esas referencias para recuperar los IDs de las tareas elegidas sin copiar listas en cada combinación.

2.5 Análisis de complejidad y Teorema Maestro

Sea C(n) el costo de resolver n tareas y S el tamaño de un frente. El costo de combinar es el número de pares que se prueban: |izq| × |der|. La recurrencia es:

> **C(n) = 2 · C(n/2) + f(n)**, con f(n) = costo de combinar ≈ |izq| · |der|

Para aplicar el Teorema Maestro (a = 2 subproblemas, b = 2 porque cada uno tiene la mitad, $n^{\log_b a} = n$) hay que saber cómo crece f(n). El tamaño de un frente está limitado por la suma de los tiempos de sus tareas, que crece proporcional al número de tareas, y por T + 1:

Escenario	f(n)	Caso del Teorema Maestro	Resultado
Nuestras instancias (T proporcional a n; el frente crece ~ n)	$\Theta(n^2)$	Caso 3: $f(n) = \Omega(n^{1+\varepsilon})$ con $\varepsilon = 1$, y regularidad $2 \cdot f(n/2) = n^2/2 \le \frac{1}{2} \cdot f(n)$	**$\Theta(n^2)$**
Cota general (T fijo, frente ≤ T+1)	$O(T^2)$, no crece con n	Caso 1: $f(n) = O(n^{1-\varepsilon})$	$O(n \cdot T^2)$
Sin podar por T ni por dominancia	$\Theta(2^{n/2} \cdot 2^{n/2})$	Domina el último combinar	$\Theta(2^n)$, igual a la fuerza bruta

Verificación empírica: contamos los pares probados en todas las combinaciones. Si el algoritmo es $\Theta(n^2)$, al duplicar n los pares deben multiplicarse por 4:

n	50	100	200	400	800
Pares probados	32,622	292,843	2,132,787	11,543,542	53,868,948
Factor al duplicar n	—	×9.0	×7.3	×5.4	×4.7

El factor baja hacia 4 a medida que n crece, que es lo que predice $\Theta(n^2)$. Con n pequeños es mayor porque los frentes todavía no han llegado a su tamaño “proporcional” (la poda por dominancia elimina relativamente más).

Complejidad espacial: $O(n \cdot T)$ en el peor caso. Se guardan los frentes de todos los subproblemas con sus referencias para reconstruir la solución, y la recursión tiene profundidad

$\log_2 n$.

¿Es óptima? Sí. Solo descarta opciones que se pasan de T o que están dominadas, y una opción dominada nunca puede llevar a una solución mejor que la que la domina. En todos los casos de prueba coincide con la fuerza bruta (la comparativa lo verifica con un assert).

2.6 Por qué divide y vencerás resulta la técnica exacta más lenta

Con n = 1000 tardó 30.6 s, frente a 0.94 s de la programación dinámica. La razón está en el paso de **combinar**: juntar dos frentes grandes obliga a probar todos los pares, y eso es cuadrático. En la última combinación cada mitad tiene miles de opciones, así que se prueban decenas de millones de pares. Dividir en dos mitades iguales sirve cuando combinar es barato (como en Merge Sort, que combina en O(n)); aquí combinar es caro, y conviene más agregar las tareas **de a una**, que es exactamente lo que hace la programación dinámica.

Ventaja que sí tiene: las dos mitades son independientes, así que podrían resolverse **en paralelo** en dos procesadores o servidores distintos. Ni la programación dinámica (cada fila depende de la anterior) ni el backtracking se paralelizan tan fácil.

3. SOLUCIÓN POR PROGRAMACIÓN DINÁMICA (src/dinamica.py)

3.1 Por qué aplica: subproblemas superpuestos

En el árbol de la fuerza bruta, muchas ramas distintas llegan a la misma pregunta: *"con las tareas que faltan y c unidades de tiempo libres, ¿cuál es la mejor prioridad?"*. Por ejemplo, tomar la tarea A (tiempo 5) y no la B (tiempo 5), o tomar la B y no la A, deja exactamente la misma capacidad para el resto. La fuerza bruta resuelve esa pregunta una y otra vez; la programación dinámica la **resuelve una sola vez y guarda la respuesta en una tabla**. Además el problema tiene **subestructura óptima**: la mejor solución contiene la mejor solución del subproblema que queda.

3.2 Subproblema y recurrencia

mejor[i][c] = máxima prioridad usando solo las primeras *i* tareas con *c* unidades de tiempo.

mejor[0][c] = 0 (sin tareas no hay prioridad)
mejor[i][c] = mejor[i−1][c] si $t_i > c$ (la tarea no cabe)
mejor[i][c] = max(mejor[i−1][c] , mejor[i−1][c − t_i] + p_i) (no tomarla / tomarla)

Respuesta: mejor[n][T]

3.3 Ejemplo: la tabla del contraejemplo (T = 10)

i \ c	0	1	2	3	4	5	6	7	8	9	10
0 (ninguna)	0	0	0	0	0	0	0	0	0	0	0
1: (6, 7)	0	0	0	0	0	0	7	7	7	7	7
2: (5, 5)	0	0	0	0	0	5	7	7	7	7	7
3: (5, 5)	0	0	0	0	0	5	7	7	7	7	10

Ejemplo de una celda: mejor[3][10] = max(mejor[2][10], mejor[2][10−5] + 5) = max(7, 5 + 5) = **10**. **Reconstrucción** (de abajo hacia arriba): en la fila 3, c = 10 el valor cambió respecto a la fila 2, así que la tarea 3 se tomó y quedan c = 10 − 5 = 5. En la fila 2, c = 5 también cambió (5 vs 0), así que se tomó la tarea 2 y quedan 0. La tarea 1 no se tomó. Resultado: tareas {2, 3}, prioridad 10.

3.4 Implementación (núcleo)

```
def programacion_dinamica(tareas, capacidad):
    mejor = [0] * (capacidad + 1)          # una fila de la tabla
    tomada = []                            # para reconstruir la solucion
    for tarea in tareas:
        t, p = tarea.tiempo, tarea.prioridad
        fila = bytearray(capacidad + 1)
        for c in range(capacidad, t - 1, -1):     # de mayor a menor
            if mejor[c - t] + p > mejor[c]:
                mejor[c] = mejor[c - t] + p
                fila[c] = 1                        # convino tomar la tarea
        tomada.append(fila)
    ... recorrer 'tomada' desde la ultima tarea para obtener los IDs
    return mejor[capacidad], seleccionadas
```

Detalle importante: solo se guarda **una fila** de valores y se recorre c **de mayor a menor**. Así, cuando se lee mejor[c − t], ese valor todavía corresponde a la fila anterior (sin la tarea actual) y una tarea no se puede usar dos veces. Para saber *qué* tareas se eligieron se guarda por cada tarea una fila de bytes (0/1).

3.5 Complejidad

- **Temporal: O(n · T).** Se llenan n × (T + 1) celdas y cada una cuesta O(1): una resta, una suma y una comparación. Con n = 1000 y T = 7636 son 7.6 millones de celdas, que se calculan en ~0.94 s.
- **Espacial: O(n · T)** para poder reconstruir la solución (7.6 MB con n = 1000). Si solo se quisiera el valor óptimo bastaría O(T).
- **Pseudo-polinomial:** es polinomial en n y en el *valor* de T, no en la cantidad de dígitos de T. Si los tiempos se midieran en milisegundos (T ≈ 7.6 millones) la tabla tendría 7.6 mil millones de celdas y sería inviable. Esto no contradice que el problema sea NP-difícil.
- **Óptima:** sí. Evalúa todos los subproblemas posibles; nunca descarta nada que pueda servir.
- **Relación con divide y vencerás:** PD es una “división” en *una tarea + el resto* donde los subproblemas se repiten y se guardan. Combinar una sola tarea con la fila anterior cuesta O(T), en vez del O(S^2) de combinar dos mitades.

4. COMPARATIVA FINAL DE LAS 5 TÉCNICAS

Todas las mediciones salen de `python src/comparativa.py`, ejecutado en un mismo equipo. Los tiempos absolutos cambian entre computadores, pero las proporciones entre técnicas se mantienen. El script verifica en cada instancia que las cuatro técnicas exactas den la misma prioridad.

4.1 Tabla comparativa final

Técnica	Complejidad temporal	Complejidad espacial	¿Solución óptima?	Tiempo real (n=100)	Tiempo real (n=1000)
Fuerza bruta	$O(2^n)$	O(n)	Sí	Inviable ($\sim 10^{15}$ años, estimado)	Inviable ($\sim 10^{286}$ años, estimado)
Greedy	O(n log n)	O(n)	No (99.57 % y 99.97 %)	0.000031 s	0.000350 s
Backtracking	$O(n \cdot 2^n)$ peor caso	O(n)	Sí	0.000385 s	2.58 s (varía de 0.03 a 30 s según los datos)
Divide y Vencerás	$\Theta(n^2)$ en estas instancias; $O(n \cdot T^2)$ general	O(n·T)	Sí	0.063 s	30.64 s
Programación dinámica	O(n·T)	O(n·T)	Sí	0.0075 s	0.94 s

n = 100 → T = 775 (caso_n100.txt). n = 1000 → T = 7636 (caso_n1000.txt). La fuerza bruta se estima a partir de su tiempo medido con n = 28 (46.05 s), duplicándolo por cada tarea adicional (crecimiento 2^n).

4.2 Resultados en los archivos de prueba

Archivo	n	Óptimo	FB	Greedy (prioridad)	BT	DV	PD
contraej.	3	10	0.00001 s	0.00001 s (7) ✗	0.0002 s	0.00002 s	0.00001 s
pequeño	10	90	0.0002 s	0.00001 s (85) ✗	0.0001 s	0.0001 s	0.0001 s
mediano	20	255	0.303 s	0.00004 s (238) ✗	0.0001 s	0.0010 s	0.0005 s
grande	28	378	46.05 s	0.00003 s (362) ✗	0.0002 s	0.0013 s	0.0005 s
n100	100	1173	inviable	0.00003 s (1168) ✗	0.0004 s	0.063 s	0.0075 s
n1000	1000	11843	inviable	0.0004 s (11840) ✗	2.58 s	30.64 s	0.94 s

FB, BT, DV y PD obtuvieron exactamente el óptimo en todos los casos. ✗ = el greedy no alcanzó el óptimo.

4.3 Gráficas de tiempos de las 5 soluciones

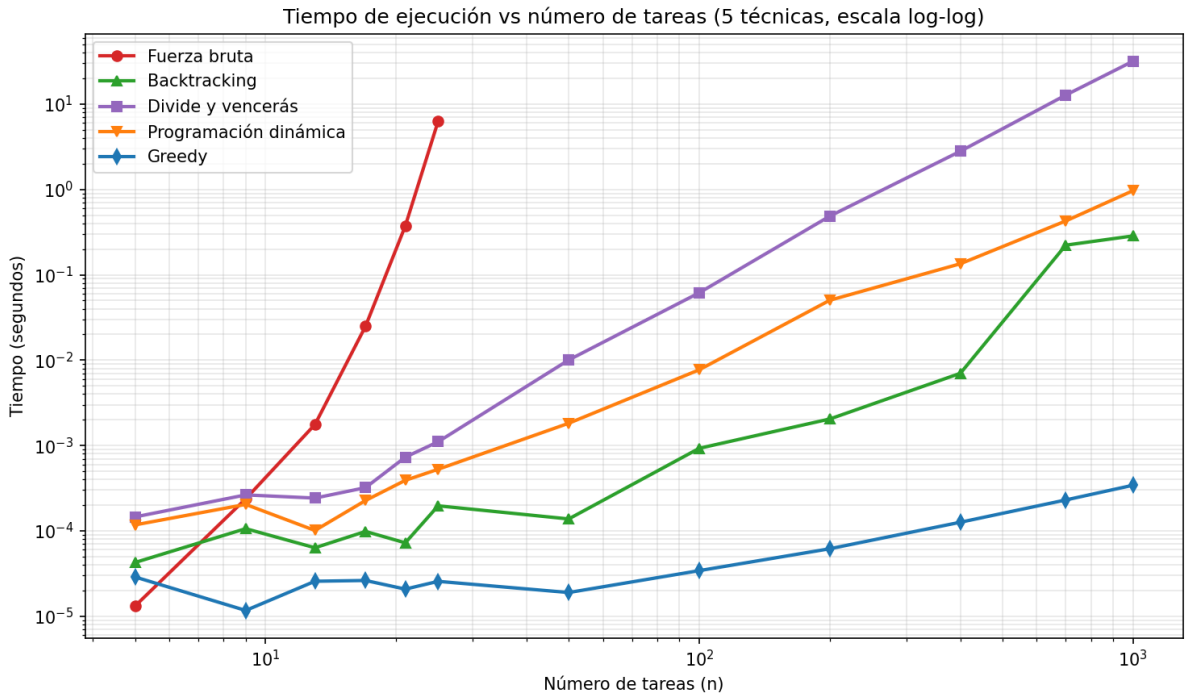


Figura 1. Tiempo vs n (log-log) con instancias generadas por la misma función (semilla 5000 + n). En escala log-log una recta indica crecimiento polinomial y su pendiente es el exponente. Entre n = 100 y n = 1000, PD tiene pendiente ≈ 2.1 (O(n·T) con T proporcional a n) y DV ≈ 2.7 (la teoría predice que baje a 2 cuando n crece; ver 2.5), con una constante mucho mayor. La fuerza bruta no es una recta: se dispara de forma exponencial.

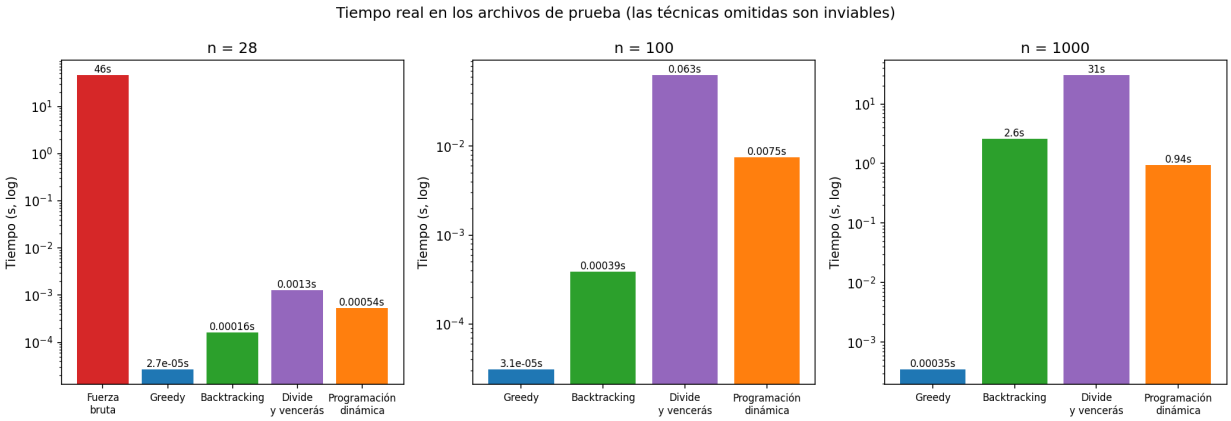


Figura 2. Tiempos reales en los archivos con n = 28, 100 y 1000 (escala logarítmica). Las técnicas que no aparecen no se pueden ejecutar en ese tamaño.

4.4 Experimento complementario: ¿qué tan predecible es cada técnica?

Con un solo caso de n = 1000 el backtracking pareció razonable (2.58 s). Para comprobar si eso es estable, `src/experimento_variabilidad.py` ejecuta backtracking y programación dinámica sobre 9 instancias distintas, todas con n = 1000 y generadas igual:

Semilla	1001	1003	1004	1005	1006	1007	1008	1009	1011
BT (s)	1.82	**29.78**	0.13	0.03	**20.15**	0.96	2.62	0.32	0.05
BT nodos	1.6 M	40.6 M	139 k	1.5 k	29.2 M	1.3 M	3.6 M	394 k	34 k
PD (s)	1.53	0.96	0.95	0.92	0.93	0.98	0.93	0.92	0.94

El backtracking varió **unas 1000 veces** (de 0.03 s a 29.8 s) con el mismo tamaño, porque su tiempo depende de qué tan bien poda la cota en esos datos concretos. La programación dinámica siempre llena la misma tabla de n × (T + 1) celdas y se mantuvo entre 0.92 s y 1.53 s. Esto confirma en la práctica lo que dice el análisis teórico: el backtracking es exponencial en el peor caso y la PD es O(n·T) siempre.

4.5 Análisis: ¿qué técnica es la más adecuada para este problema y por qué?

Recomendación: Programación Dinámica. Es la única técnica que cumple a la vez las tres cosas que necesita un servidor real: (1) **siempre da el óptimo**, (2) su tiempo es **predecible** (depende solo de n y T, no de los datos) y (3) es **rápida** para tamaños realistas: 1000 tareas en menos de 1 segundo.

Técnica	Cuándo SÍ usarla en este problema	Cuándo NO
Fuerza bruta	Para validar a las demás con casos pequeños (n ≤ 25).	Con más de ~30 tareas: cada tarea extra duplica el tiempo.
Greedy	Cuando T es enorme (la tabla de PD no cabe en memoria) o se necesita respuesta en microsegundos. Pierde muy poco con muchas tareas (99.97 % con n = 1000).	Cuando hay pocas tareas grandes respecto a T: perdió hasta el 30 % (contraejemplo) y el 7 % en el caso mediano.
Backtracking	Cuando se necesita el óptimo y la cota poda bien; muchas veces es el más rápido de los exactos.	Cuando se necesita un tiempo garantizado: puede tardar 1000 veces más con datos parecidos.
Divide y vencerás	Si se pudiera repartir el trabajo en paralelo (las mitades son independientes).	En un solo procesador: combinar frentes es cuadrático y es 30 veces más lento que PD.
Programación dinámica	Casi siempre: tiempos enteros y T moderado (miles o cientos de miles).	Si T es gigantesco (tiempos en milisegundos o con decimales): la tabla no cabe.

Trade-offs en resumen: la más rápida es el **greedy**; las que dan el mejor resultado son **FB, BT, DV y PD**; la más fácil de implementar es el **greedy** (ordenar y un ciclo) seguido de la fuerza bruta; la más difícil de diseñar fue **divide y vencerás**, porque hubo que descubrir que cada mitad debe devolver un frente de opciones y no un solo número.

5. CONCLUSIONES

5.1 Qué aprendimos al aplicar cada técnica al mismo problema

- **Fuerza bruta:** es la referencia de “lo correcto”, pero su crecimiento 2^n no es un detalle teórico: pasar de 20 a 28 tareas llevó el tiempo de 0.3 s a 46 s. La usamos durante todo el proyecto para comprobar que las demás técnicas daban bien.
- **Greedy:** una decisión local razonable (lo que más rinde por unidad de tiempo) no garantiza el óptimo global, y un contraejemplo de solo 3 tareas basta para demostrarlo. Aun así, con muchas tareas el error es casi nulo.
- **Backtracking:** la poda por capacidad casi no ayudó; lo que redujo 203 millones de nodos a 146 fue la **cota**. Aprendimos que una buena cota vale más que muchas reglas, y que un buen promedio no significa un buen peor caso.

- **Divide y vencerás:** no todo problema se beneficia de partirse en mitades. Lo importante es cuánto cuesta combinar: aquí combinar es cuadrático, mientras que en Merge Sort es lineal. Fue la técnica que más nos hizo pensar en *qué* debe devolver un subproblema.
- **Programación dinámica:** reconocer que los subproblemas se repiten cambió un problema exponencial en uno de ~1 segundo para 1000 tareas. También vimos su límite: depende del valor de T.

5.2 ¿En qué escenarios reales usaríamos cada paradigma?

Paradigma	Escenarios reales
Fuerza bruta	Pruebas y validación de otros algoritmos; problemas diminutos; descifrar contraseñas cortas por intentos (lo que motiva usar contraseñas largas).
Greedy	Planificadores de tareas en tiempo real; algoritmo de Dijkstra para rutas en GPS; compresión Huffman; dar vueltas con el menor número de monedas.
Backtracking	Sudoku y puzzles; asignación de horarios con restricciones (salones, profesores); problemas de N reinas; solucionadores de restricciones.
Divide y vencerás	Ordenamiento (Merge Sort, Quick Sort); búsqueda binaria; FFT en audio e imágenes; procesamiento distribuido tipo MapReduce donde cada parte va a un servidor.
Programación dinámica	Asignación de presupuesto o recursos limitados (nuestro caso); corrector ortográfico (distancia de edición); comparación de ADN; rutas más cortas en mapas.

5.3 Limitaciones encontradas

- **Modelo simplificado:** trabajamos con un solo servidor y sin dependencias entre tareas. Con m servidores el problema pasa a ser una mochila múltiple, y con dependencias habría que respetar un orden (grafo dirigido). Las cinco técnicas se pueden extender, pero el costo crece: por ejemplo, la PD necesitaría una dimensión de capacidad por cada servidor.
- **Datos sintéticos:** las instancias son generadas con una distribución fija (tiempos de 5 a 30, T = 45 %). Con otras distribuciones los resultados del backtracking y del greedy pueden cambiar bastante (como mostró el experimento de variabilidad).
- **Mediciones en Python y en un solo equipo:** los tiempos absolutos dependen del computador y del intérprete; por eso también medimos nodos, pares y celdas, que no dependen de la máquina.
- **Tiempos de la fuerza bruta para n = 100 y n = 1000:** son estimaciones por extrapolación; ejecutarla es imposible.
- **Programación dinámica pseudo-polinomial:** si T fuera muy grande, la técnica recomendada dejaría de servir y habría que usar greedy o backtracking.

6. EJECUCIÓN, ESTRUCTURA Y REFERENCIAS

6.1 Cómo ejecutar

```
pip install matplotlib                      # unica dependencia externa
python src/fuerza_bruta.py                  # Entrega 1 (el caso grande tarda ~30-50 s)
python src/greedy.py                        # Entrega 2
python src/backtracking.py                  # Entrega 2
python src/divide_y_venceras.py             # Entrega 3 (n=1000 tarda ~30 s)
python src/dinamica.py                      # Entrega 3
python src/comparativa.py                   # comparativa final completa (~3 min)
python src/comparativa.py --rapido          # version para la demo (~20 s)
python src/experimento_variabilidad.py      # backtracking vs PD en 9 instancias n=1000
python src/generar_datos.py                 # regenera los casos generados
```

6.2 Estructura del repositorio

```
datos/        6 casos de prueba (contraejemplo, pequeno, mediano, grande, n100, n1000)
src/          fuerza_bruta.py, greedy.py, backtracking.py, divide_y_venceras.py, dinamica.py,
              comparativa.py, experimento_variabilidad.py, generar_datos.py
docs/         entrega1.pdf, entrega2.pdf, entrega3_final.pdf y graficas de cada entrega
presentacion/ presentacion_final.pdf
```

6.3 Referencias

- Cormen, T., Leiserson, C., Rivest, R. y Stein, C. (2022). *Introduction to Algorithms* (4.ª ed.). MIT Press. Teorema Maestro y programación dinámica.
- Kleinberg, J. y Tardos, É. (2006). *Algorithm Design*. Pearson. Capítulo 6: mochila y programación dinámica.
- Horowitz, E. y Sahni, S. (1974). Computing partitions with applications to the knapsack problem. *Journal of the ACM*, 21(2), 277–292. Base de la estrategia de dividir las tareas en dos mitades y combinar sus listas de opciones.
- Kellerer, H., Pferschy, U. y Pisinger, D. (2004). *Knapsack Problems*. Springer. Cotas superiores (relajación fraccional) y listas de dominancia.